Programming in C

Lecture Notes for B.Tech Students

Unit 1	Strings in C: gets/puts, strlen, strcpy, strcmp
Unit 2	Pointers: addresses and pointer variables
Unit 3	Structures: creation, size and padding, typedef
Unit 4	File Handling: fopen, read/write, binary files, fseek

All programs in these notes have been tested with GCC. Sizes quoted assume a typical 64-bit system (char = 1 byte, int = 4, float = 4, double = 8, pointer = 8).

Unit 1: Strings in C

1.1 What is a String?

C has no separate **string** data type. A string in C is simply an **array of characters terminated by a special null character** `'\0'` (ASCII value 0). This terminator is how every string function knows where the string ends. If the `'\0'` is missing, functions keep reading past the end of the array into random memory.

```
char name[10] = "RAVI";                     /* compiler adds '\0' automatically */
char city[]   = "Delhi";                    /* size decided by compiler: 6 bytes */
char word[5]  = {'H', 'E', 'L', 'L', '\0'}; /* manual initialization */
```

Memory layout of char name[10] = "RAVI";

R	A	V	I	\0	?	?	?	?	?
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

> **Remember**
>
> To store a string of **n** characters you need an array of at least **n + 1** characters; one extra byte is for `'\0'`. "Delhi" has 5 letters but occupies 6 bytes.

Character vs string constant: `'A'` (single quotes) is a single character occupying 1 byte. `"A"` (double quotes) is a string occupying 2 bytes: 'A' and '\0'.

1.2 Reading and Printing Strings

Using scanf() and printf() with %s

`scanf("%s", str)` reads characters only up to the first **whitespace** (space, tab or newline). Note that no & is needed, because the array name itself is already an address.

```
#include <stdio.h>
int main(void) {
    char name[30];
    printf("Enter your full name: ");
    scanf("%29s", name);
    printf("Hello, %s\n", name);
    return 0;
}
```

```
Output:
Enter your full name: Rahul Kumar Sharma
Hello, Rahul
```

Only **"Rahul"** was stored. The remaining text " Kumar Sharma" stays in the input buffer and may get picked up by the next input statement, causing confusing bugs. This is the main limitation of `scanf("%s")`.

Using gets() and puts()

gets(str) reads an entire line, **including spaces**, until the user presses Enter. The newline is discarded and replaced with `'\0'`. **puts(str)** prints a string and **automatically adds a newline** at the

end. Both are declared in <stdio.h>.

```
char name[30];
printf("Enter your full name: ");
gets(name);          /* reads "Rahul Kumar Sharma" completely */
puts(name);          /* prints it and moves to the next line */
```

```
Output:
Enter your full name: Rahul Kumar Sharma
Rahul Kumar Sharma
```

Why gets()/puts() are preferred over scanf()/printf() for strings

Point	scanf("%s") / printf("%s")	gets() / puts()
Spaces in input	Stops reading at the first space, so multi-word input is cut off.	Reads the whole line including spaces.
Newline on output	Must write \n yourself.	puts() adds \n automatically.
Simplicity	Needs a format string (%s).	Just pass the string; no format specifier.
Speed / overhead	Must parse the format string at runtime.	Dedicated to strings, so slightly lighter.
Leftover input	Rest of the line remains in the buffer.	Consumes the whole line including Enter.

Very important: gets() is unsafe and removed from modern C

`gets()` has no way to know the size of the array. If the user types more characters than the array can hold, it writes beyond the array (a **buffer overflow**), corrupting memory and creating security holes. For this reason it was deprecated in C99 and **removed entirely in C11**. Modern compilers warn about it or refuse to compile it. In exams you should know gets(), but in real programs always use **fgets()**, which gives the same 'read a whole line' advantage safely.

The safe replacement: fgets()

Syntax: `fgets(str, size, stdin);` It reads at most **size - 1** characters, so it can never overflow. One difference: fgets() **keeps the newline** in the string, which we usually remove.

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char name[30];
    printf("Enter your full name: ");
    fgets(name, sizeof(name), stdin);
    name[strcspn(name, "\n")] = '\0';   /* remove trailing newline */
    puts(name);
    return 0;
}
```

```
Output:
Enter your full name: Rahul Kumar Sharma
Rahul Kumar Sharma
```

1.3 String Library Functions (string.h)

To use these functions write `#include <string.h>` at the top of the program.

(a) strlen(): length of a string

Syntax: `size_t strlen(const char *str);` It returns the number of characters **before** the `'\0'`. The null character is **not** counted.

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[20] = "Hello World";
    printf("strlen = %zu\n", strlen(s));   /* characters in use        */
    printf("sizeof = %zu\n", sizeof(s));   /* total size of the array  */
    return 0;
}
```

```
Output:
strlen = 11
sizeof = 20
```

> **strlen() vs sizeof**
>
> **strlen** is a function that counts characters at runtime up to '\0'. **sizeof** is an operator that gives the memory size of the whole array at compile time, including unused bytes and the '\0'.

How strlen works internally (a favourite exam question: write your own strlen):

```
int my_strlen(const char s[]) {
    int i = 0;
    while (s[i] != '\0')
        i++;
    return i;
}
```

(b) strcpy(): copy one string into another

Syntax: `strcpy(destination, source);` It copies every character of source, **including the '\0'**, into destination. Arrays cannot be assigned with = in C, so `dest = src;` is a compile error; strcpy is the correct way.

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char src[] = "Programming";
    char dest[20];
    /* dest = src;   ERROR: array type is not assignable */
    strcpy(dest, src);
    printf("Source      : %s\n", src);
    printf("Destination : %s\n", dest);
    return 0;
}
```

```
Output:
Source      : Programming
Destination : Programming
```

> **Caution**
>
> The destination must be large enough to hold the source plus '\0'. strcpy() does no size checking, so a small destination causes buffer overflow. A safer variant is `strncpy(dest, src, n)`, which copies at most n characters.

(c) strcmp(): compare two strings

Syntax: `int strcmp(const char *s1, const char *s2);` It compares the two strings character by character using ASCII values and stops at the first difference.

Return value	Meaning	Example
0	Both strings are identical	strcmp("abc", "abc") gives 0
Negative (< 0)	s1 comes before s2 (dictionary order)	strcmp("apple", "mango") < 0
Positive (> 0)	s1 comes after s2	strcmp("zoo", "cat") > 0

The exact value returned for unequal strings depends on the compiler, so always test for `== 0`, `< 0` or `> 0`, never for a specific number like -1. Comparison is **case-sensitive**: "Apple" and "apple" differ because 'A' is 65 and 'a' is 97.

> **Why not use == to compare strings?**
>
> `if (s1 == s2)` compares the **addresses** of the two arrays, not their contents. Two different arrays always have different addresses, so this is almost always false even if the text matches. Always use strcmp().

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char password[] = "c@2026";
    char input[20];
    printf("Enter password: ");
    scanf("%19s", input);
    if (strcmp(input, password) == 0)
        printf("Access granted\n");
    else
        printf("Wrong password\n");
    return 0;
}
```

```
Output:
Enter password: c@2026
Access granted
```

Other useful string.h functions (for reference)

Function	Purpose
`strcat(s1, s2)`	Appends s2 at the end of s1
`strncpy(d, s, n)`	Copies at most n characters
`strncmp(s1, s2, n)`	Compares only the first n characters

Function	Purpose
`strchr(s, ch)`	Returns pointer to first occurrence of ch in s
`strstr(s1, s2)`	Returns pointer to first occurrence of s2 in s1

Complete program combining all functions

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char a[50], b[50], copy[50];

    printf("Enter first string : ");
    fgets(a, sizeof a, stdin);  a[strcspn(a, "\n")] = '\0';
    printf("Enter second string: ");
    fgets(b, sizeof b, stdin);  b[strcspn(b, "\n")] = '\0';

    printf("Length of first  = %zu\n", strlen(a));
    printf("Length of second = %zu\n", strlen(b));

    strcpy(copy, a);
    printf("Copy of first    = ");
    puts(copy);

    int r = strcmp(a, b);
    if (r == 0)      puts("Strings are equal");
    else if (r < 0)  puts("First string comes first");
    else             puts("Second string comes first");
    return 0;
}
```

```
Output:
Enter first string : good morning
Enter second string: good night
Length of first  = 12
Length of second = 10
Copy of first    = good morning
First string comes first
```

Practice / Viva Questions

1. Why does a string need one extra byte? What happens if '\0' is missing?
2. Explain with an example why scanf("%s") cannot read "New Delhi" completely.
3. Why was gets() removed in C11? What should be used instead?
4. Differentiate between strlen() and sizeof for a character array.
5. Why can we not compare two strings using ==?
6. Write your own versions of strcpy() and strcmp() without using string.h.

Unit 2: Pointers

2.1 Concept of Memory and Address

The computer's memory (RAM) can be imagined as a long row of boxes, each one byte in size. Every box has a unique number called its **address**. When you declare a variable, the compiler reserves some of these boxes for it. The variable therefore has two things associated with it: its **value** (what is stored in it) and its **address** (where it is stored).

```
int marks = 85;
```

Property	Value
Name of variable	marks
Value stored	85
Address (example)	0x7ffd5a10 (actual value differs on each run)
Size	4 bytes (int)

The address-of operator (&)

The unary operator & gives the address of a variable. Addresses are printed using the %p format specifier. You have already used it in `scanf("%d", &x)`: scanf needs the address so it knows where to store the value.

```
#include <stdio.h>
int main(void) {
    int marks = 85;
    printf("Value   of marks = %d\n", marks);
    printf("Address of marks = %p\n", (void *)&marks);
    return 0;
}
```

```
Output:
Value   of marks = 85
Address of marks = 0x7ffd5a10c3ac
```

2.2 Pointer Variables

A **pointer** is a variable that **stores the address of another variable**. Just as an int variable holds an integer, a pointer variable holds an address.

Declaration

```
data_type *pointer_name;

int    *p;     /* p can hold the address of an int    */
float  *fp;    /* fp can hold the address of a float  */
char   *cp;    /* cp can hold the address of a char   */
```

The * in the declaration means "pointer to". The data type tells the compiler what kind of value lives at that address, so it knows how many bytes to read and how to interpret them.

Initialization and the dereference operator (*)

Once a pointer holds an address, the **dereference** (indirection) operator `*` is used to access the value stored at that address. Read `*p` as "the value at address p".

```
#include <stdio.h>
int main(void) {
    int x = 10;
    int *p = &x;          /* p now points to x */

    printf("x      = %d\n", x);
    printf("&x     = %p\n", (void *)&x);
    printf("p      = %p\n", (void *)p);     /* same as &x            */
    printf("*p     = %d\n", *p);            /* value at address p    */
    printf("&p     = %p\n", (void *)&p);    /* p has its own address */

    *p = 25;              /* changes x through the pointer */
    printf("x now  = %d\n", x);
    return 0;
}
```

```
Output:
x      = 10
&x     = 0x7ffc1a2b3c4c
p      = 0x7ffc1a2b3c4c
*p     = 10
&p     = 0x7ffc1a2b3c50
x now  = 25
```

Picture of memory for the above program:

*p stores the address of x, so p "points to" x. Changing *p changes x.*

Expression	Meaning	Value in example
x	value of x	10
&x	address of x	0x...3c4c
p	value of p = address of x	0x...3c4c
*p	value at the address stored in p	10
&p	address of the pointer itself	0x...3c50

> **Two meanings of ***
>
> In a **declaration** (`int *p;`) the star means "p is a pointer". In an **expression** (`*p = 5;`) the star means "go to the address in p". Students often confuse the two.

2.3 Size of a Pointer

A pointer stores an address, and all addresses on a machine have the same size. Therefore every pointer has the **same size regardless of the data type it points to**: 8 bytes on a 64-bit system and 4 bytes on a 32-bit system.

```
#include <stdio.h>
int main(void) {
    printf("%zu %zu %zu\n", sizeof(int *), sizeof(char *), sizeof(double *));
    return 0;
}
```

```
Output:
8 8 8
```

2.4 NULL Pointer and Wild Pointer

- A **NULL pointer** points to nothing: `int *p = NULL;` It is good practice to initialize pointers to NULL and check `if (p != NULL)` before using them.
- A **wild pointer** is an uninitialized pointer: `int *p; *p = 5;` It contains a garbage address, and writing to it can crash the program (segmentation fault).
- A **dangling pointer** points to memory that no longer exists, for example the address of a local variable of a function that has already returned.

2.5 Pointer Arithmetic

When you add 1 to a pointer, it moves forward by **sizeof(data type)** bytes, not by 1 byte. So for an `int *p` holding address 1000, `p + 1` is 1004. This is what makes pointers so natural for walking through arrays.

```
#include <stdio.h>
int main(void) {
    int arr[4] = {10, 20, 30, 40};
    int *p = arr;                 /* array name = address of first element */
    for (int i = 0; i < 4; i++)
        printf("*(p + %d) = %d\n", i, *(p + i));
    return 0;
}
```

```
Output:
*(p + 0) = 10
*(p + 1) = 20
*(p + 2) = 30
*(p + 3) = 40
```

This shows the key relationship `arr[i]` is identical to `*(arr + i)`. It also explains why a string (char array) is passed to functions such as strlen() as a `char *`.

2.6 Application: Call by Reference

Normally C passes arguments **by value**: the function gets a copy, so changes do not affect the caller. By passing **addresses**, a function can modify the caller's variables. The classic example is swapping:

```c
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main(void) {
    int x = 5, y = 9;
    swap(&x, &y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

```
Output:
x = 9, y = 5
```

Practice / Viva Questions

1. What is a pointer? How is it different from an ordinary variable?
2. Explain the & and * operators with an example and a memory diagram.
3. Why is the size of int*, char* and double* the same?
4. If int *p holds 2000, what is p + 3? Justify.
5. What is a NULL pointer? What is a wild pointer?
6. Write a function using pointers that finds both the sum and the product of two numbers.

Unit 3: Structures

3.1 Need for Structures

An array stores many values of the **same** type. Real-world entities, however, have properties of **different** types. A student has a roll number (int), a name (string) and a CGPA (float). A **structure** is a user-defined data type that groups related variables of different types under one name.

3.2 Defining a Structure

```
struct structure_name {
    data_type member1;
    data_type member2;
    ...
};                          /* the semicolon is compulsory */
```

```
struct Student {
    int   roll;
    char  name[30];
    float cgpa;
};
```

Defining a structure only creates a **template** (a blueprint). No memory is allocated until a variable of that structure type is declared.

Declaring structure variables

```
struct Student s1, s2;                /* method 1: separately           */

struct Point { int x, y; } p1, p2;    /* method 2: along with definition */
```

Initialization and accessing members

Members are accessed with the **dot operator (.)**: `variable.member`.

```
#include <stdio.h>
#include <string.h>

struct Student {
    int   roll;
    char  name[30];
    float cgpa;
};

int main(void) {
    struct Student s1 = {101, "Ananya", 8.7f};   /* initialization at declaration */
    struct Student s2;

    s2.roll = 102;                               /* member-wise assignment */
    strcpy(s2.name, "Vikram");                   /* strings need strcpy    */
    s2.cgpa = 9.1f;

    printf("%d  %-8s %.1f\n", s1.roll, s1.name, s1.cgpa);
    printf("%d  %-8s %.1f\n", s2.roll, s2.name, s2.cgpa);
    return 0;
}
```

```
Output:
101  Ananya   8.7
102  Vikram   9.1
```

> **Note**
>
> Unlike arrays, a whole structure can be copied with assignment: `s2 = s1;` copies every member. However, structures cannot be compared with ==; compare member by member.

Array of structures

```
#include <stdio.h>
struct Student { int roll; char name[30]; float cgpa; };

int main(void) {
    struct Student cls[3];
    for (int i = 0; i < 3; i++) {
        printf("Enter roll, name, cgpa: ");
        scanf("%d %29s %f", &cls[i].roll, cls[i].name, &cls[i].cgpa);
    }
    printf("\nStudents with CGPA above 8:\n");
    for (int i = 0; i < 3; i++)
        if (cls[i].cgpa > 8.0f)
            printf("%d %s\n", cls[i].roll, cls[i].name);
    return 0;
}
```

```
Output:
Enter roll, name, cgpa: 1 Riya 8.4
Enter roll, name, cgpa: 2 Aman 7.2
Enter roll, name, cgpa: 3 Kabir 9.0

Students with CGPA above 8:
1 Riya
3 Kabir
```

Pointer to structure and the arrow operator (->)

When you have a pointer to a structure, members are accessed with `->`. The expression `ptr->roll` is shorthand for `(*ptr).roll`.

```
struct Student s = {101, "Ananya", 8.7f};
struct Student *ptr = &s;
printf("%d %s\n", ptr->roll, ptr->name);    /* same as (*ptr).roll */
```

3.3 Size of a Structure

You might expect the size of a structure to be the sum of the sizes of its members. Often it is **larger**, because of **structure padding**.

Why padding happens: alignment

The processor reads memory fastest when each data item starts at an address that is a multiple of its size: an int (4 bytes) at an address divisible by 4, a double (8 bytes) at an address divisible by 8. To guarantee this, the compiler inserts unused **padding bytes** between members, and also at the end

so the total size is a multiple of the largest member's alignment (this keeps every element of an array of structures aligned).

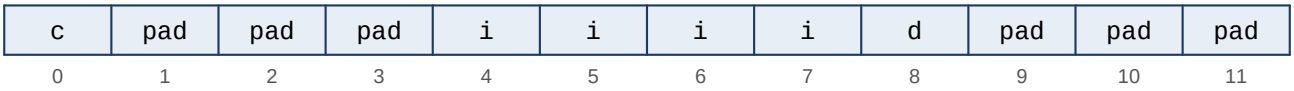
```
#include <stdio.h>

struct A { char c;  int i;  char d; };      /* poor ordering  */
struct B { int i;   char c; char d; };      /* better ordering */
struct C { char c;  double x; };

int main(void) {
    printf("sizeof(struct A) = %zu\n", sizeof(struct A));
    printf("sizeof(struct B) = %zu\n", sizeof(struct B));
    printf("sizeof(struct C) = %zu\n", sizeof(struct C));
    return 0;
}
```

```
Output:
sizeof(struct A) = 12
sizeof(struct B) = 8
sizeof(struct C) = 16
```

Layout of struct A (sum of members = 1 + 4 + 1 = 6, but actual size = 12):



Layout of struct B (same members, reordered; size = 8):

> **Exam tip: calculating structure size by hand**
>
> 1. Place members in the order written. 2. Before each member, add padding so its offset is a multiple of its size. 3. At the end, add padding so the total is a multiple of the largest member's size. Arranging members from **largest to smallest** usually minimizes padding.

Structure	Sum of members	Actual sizeof
`struct { char c; int i; char d; }`	6	12
`struct { int i; char c; char d; }`	6	8
`struct { char c; double x; }`	9	16
`struct { int roll; char name[30]; float cgpa; }`	38	40

3.4 typedef: Creating an Alias

Writing `struct Student` everywhere is lengthy. The **typedef** keyword creates a new name (alias) for an existing type. It does not create a new type; it only gives an existing one a shorter or more meaningful name.

```
typedef existing_type new_name;

typedef unsigned int  uint;      /* uint a;  same as unsigned int a; */
typedef float         Marks;     /* Marks m; same as float m;        */
```

typedef with structures: three common styles

```
/* Style 1: define first, alias later */
struct Student { int roll; char name[30]; };
typedef struct Student Student;

/* Style 2: combined */
typedef struct Employee {
    int   id;
    float salary;
} Employee;

/* Style 3: anonymous structure (no tag) */
typedef struct {
    int x, y;
} Point;
```

```
#include <stdio.h>

typedef struct {
    int   id;
    char  name[30];
    float salary;
} Employee;

void display(Employee e) {
    printf("ID: %d | Name: %s | Salary: %.2f\n", e.id, e.name, e.salary);
}

int main(void) {
    Employee e1 = {1, "Meera", 55000.0f};   /* no 'struct' keyword needed */
    Employee *ep = &e1;
    ep->salary += 5000;                     /* modify through pointer */
    display(e1);
    return 0;
}
```

```
Output:
ID: 1 | Name: Meera | Salary: 60000.00
```

> **typedef vs #define**
>
> `typedef` is handled by the compiler and understands types. `#define` is simple text replacement by the preprocessor. For example, with `#define PTR int*`, the declaration `PTR a, b;` makes only *a* a pointer, whereas `typedef int* PTR;` makes both a and b pointers.

Practice / Viva Questions

1. Differentiate between an array and a structure.
2. Does defining a structure allocate memory? Explain.
3. What is structure padding? Why does the compiler add it?

4. Find sizeof(struct { char a; double b; int c; }) on a 64-bit system and show the layout.
5. What is the use of typedef? Rewrite a struct-based program using typedef.
6. Explain the difference between the dot (.) and arrow (->) operators.

Unit 4: File Handling

4.1 Why File Handling?

Data stored in variables lives in RAM and is **lost when the program ends**. Files store data permanently on disk, so it can be reused later, shared with other programs, and can be much larger than what we would type by hand each time.

Types of files

Text file	Binary file
Stores data as readable characters (e.g. .txt, .c, .csv)	Stores data in the same form as in memory (e.g. .dat, .exe)
The number 12345 takes 5 bytes (one per digit)	The int 12345 takes 4 bytes
Can be opened in Notepad	Not human-readable
Functions: fprintf, fscanf, fgets, fputs, fgetc, fputc	Functions: fread, fwrite

4.2 The FILE Pointer and Basic Steps

All file operations in C go through a pointer of type `FILE *`, defined in `<stdio.h>`. The FILE structure keeps track of the file's buffer, current position, and error/end-of-file status. Every file program follows four steps:

- **Declare** a file pointer: `FILE *fp;`
- **Open** the file: `fp = fopen("data.txt", "w");`
- **Read / write** using file functions.
- **Close** the file: `fclose(fp);` This flushes buffered data to disk and releases the file.

fopen() and file modes

Syntax: `FILE *fopen(const char *filename, const char *mode);` It returns **NULL** if the file cannot be opened (e.g. file not found in "r" mode, no permission). **Always check for NULL.**

Mode	Meaning	If file exists	If file does not exist
`"r"`	Read only	Opens, reads from start	Returns NULL
`"w"`	Write only	Contents are erased	Creates new file
`"a"`	Append (write at end)	Adds to the end	Creates new file
`"r+"`	Read and write	Opens from start	Returns NULL
`"w+"`	Read and write	Contents are erased	Creates new file
`"a+"`	Read and append	Reads anywhere, writes at end	Creates new file

Adding **b** to any mode ("rb", "wb", "ab", "rb+" ...) opens the file in binary mode.

> **Warning**
>
> Opening an existing file in "w" mode **destroys** its previous contents. Use "a" if you want to add data to an existing file.

4.3 Character I/O: fputc() and fgetc()

`fputc(ch, fp)` writes one character. `fgetc(fp)` reads one character and returns **EOF** (a constant, usually -1) at the end of the file. Note that the variable receiving fgetc() must be an **int**, not char, so that EOF can be distinguished from valid characters.

```
#include <stdio.h>
int main(void) {
    FILE *fp = fopen("demo.txt", "w");
    if (fp == NULL) { printf("Cannot open file\n"); return 1; }
    char msg[] = "Hello File!";
    for (int i = 0; msg[i] != '\0'; i++)
        fputc(msg[i], fp);
    fclose(fp);

    fp = fopen("demo.txt", "r");
    if (fp == NULL) { printf("Cannot open file\n"); return 1; }
    int ch, count = 0;
    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
        count++;
    }
    printf("\nCharacters read: %d\n", count);
    fclose(fp);
    return 0;
}
```

```
Output:
Hello File!
Characters read: 11
```

4.4 String I/O: fputs() and fgets()

`fputs(str, fp)` writes a string (it does **not** add a newline, unlike puts). `fgets(str, n, fp)` reads one line (up to n-1 characters) and returns NULL at end of file. This is the standard way to read a text file line by line.

```c
#include <stdio.h>
int main(void) {
    FILE *fp = fopen("subjects.txt", "w");
    if (fp == NULL) return 1;
    fputs("Mathematics\n", fp);
    fputs("Physics\n", fp);
    fputs("Programming in C\n", fp);
    fclose(fp);

    char line[100];
    int n = 1;
    fp = fopen("subjects.txt", "r");
    if (fp == NULL) return 1;
    while (fgets(line, sizeof line, fp) != NULL)
        printf("Line %d: %s", n++, line);
    fclose(fp);
    return 0;
}
```

```
Output:
Line 1: Mathematics
Line 2: Physics
Line 3: Programming in C
```

4.5 Formatted I/O: fprintf() and fscanf()

These work exactly like printf() and scanf() but take a file pointer as the first argument. They are ideal for storing records of mixed types in a text file.

```c
#include <stdio.h>
int main(void) {
    FILE *fp = fopen("marks.txt", "w");
    if (fp == NULL) return 1;
    fprintf(fp, "%d %s %d\n", 1, "Riya", 88);
    fprintf(fp, "%d %s %d\n", 2, "Aman", 74);
    fprintf(fp, "%d %s %d\n", 3, "Kabir", 91);
    fclose(fp);

    int roll, marks, total = 0, n = 0;
    char name[30];
    fp = fopen("marks.txt", "r");
    if (fp == NULL) return 1;
    while (fscanf(fp, "%d %29s %d", &roll, name, &marks) == 3) {
        printf("%d %-6s %d\n", roll, name, marks);
        total += marks;
        n++;
    }
    fclose(fp);
    printf("Average = %.2f\n", (float)total / n);
    return 0;
}
```

```
Output:
1 Riya   88
2 Aman   74
3 Kabir  91
Average = 84.33
```

Correct way to detect end of file

Loop on the return value of the read function itself: fgetc() != EOF, fgets() != NULL, fscanf() == number of items expected, fread() == number of items requested. The common pattern `while (!feof(fp))` is a classic bug: feof() only becomes true *after* a read has failed, so the last record is usually processed twice.

4.6 Binary I/O: fwrite() and fread()

To save an entire structure in one go, use binary files. Syntax:

```
size_t fwrite(const void *ptr, size_t size, size_t count, FILE *fp);
size_t fread (void *ptr,       size_t size, size_t count, FILE *fp);
```

ptr is the address of the data, **size** is the size of one item, **count** is the number of items. Both return the number of items actually written or read. This combines all three units: structures, pointers (addresses) and files.

```
#include <stdio.h>

typedef struct {
    int   roll;
    char  name[30];
    float cgpa;
} Student;

int main(void) {
    Student list[3] = { {1, "Riya", 8.4f}, {2, "Aman", 7.2f}, {3, "Kabir", 9.0f} };

    FILE *fp = fopen("students.dat", "wb");
    if (fp == NULL) { perror("fopen"); return 1; }
    fwrite(list, sizeof(Student), 3, fp);        /* write whole array at once */
    fclose(fp);

    Student s;
    fp = fopen("students.dat", "rb");
    if (fp == NULL) { perror("fopen"); return 1; }
    while (fread(&s, sizeof(Student), 1, fp) == 1)
        printf("%d %-6s %.1f\n", s.roll, s.name, s.cgpa);
    fclose(fp);
    return 0;
}
```

```
Output:
1 Riya   8.4
2 Aman   7.2
3 Kabir  9.0
```

4.7 Random Access: fseek(), ftell(), rewind()

Files are normally read sequentially, but C allows jumping to any position.

Function	Purpose
`fseek(fp, offset, origin)`	Moves the position by offset bytes from origin. origin is SEEK_SET (beginning), SEEK_CUR (current position) or SEEK_END (end of file).

Function	Purpose
`ftell(fp)`	Returns the current position (in bytes from the beginning).
`rewind(fp)`	Moves back to the beginning of the file.

```
#include <stdio.h>
typedef struct { int roll; char name[30]; float cgpa; } Student;

int main(void) {
    FILE *fp = fopen("students.dat", "rb");
    if (fp == NULL) { perror("fopen"); return 1; }

    fseek(fp, 0, SEEK_END);
    long bytes = ftell(fp);
    printf("File size = %ld bytes, records = %ld\n", bytes, bytes / (long)sizeof(Student));

    Student s;
    fseek(fp, 1 * sizeof(Student), SEEK_SET);    /* jump directly to 2nd record */
    if (fread(&s, sizeof s, 1, fp) == 1)
        printf("Second record: %d %s\n", s.roll, s.name);

    rewind(fp);
    if (fread(&s, sizeof s, 1, fp) == 1)
        printf("First record : %d %s\n", s.roll, s.name);
    fclose(fp);
    return 0;
}
```

```
Output:
File size = 120 bytes, records = 3
Second record: 2 Aman
First record : 1 Riya
```

4.8 Summary of File Functions

Function	Use	Returns on end/failure
`fopen()`	Open a file	NULL
`fclose()`	Close a file	EOF
`fgetc() / fputc()`	Read / write one character	EOF
`fgets() / fputs()`	Read / write one line (string)	NULL / EOF
`fscanf() / fprintf()`	Formatted read / write	EOF or fewer items
`fread() / fwrite()`	Binary read / write of blocks	Fewer items than requested
`fseek() / ftell() / rewind()`	Random access	Non-zero / -1L
`feof() / ferror()`	Check end-of-file / error flag	Non-zero if set
`remove() / rename()`	Delete / rename a file	Non-zero on failure

Practice / Viva Questions

1. Why is file handling needed? Differentiate between text and binary files.
2. Explain all fopen() modes. What is the difference between "w" and "a"?
3. Why must the variable that stores the result of fgetc() be an int?
4. Write a program to copy the contents of one file to another.
5. Write a program to count characters, words and lines in a text file.
6. Using fwrite() and fread(), store details of n employees and display those earning above 50000.
7. Explain fseek() with all three origins. How can you find the size of a file?

End of Notes